

Análisis de Complejidad Computacional: Problemas de HackerRank (Pairs y Sherlock)

Juan José Bolaños Delgado 2617234

19 de junio de 2026

Resumen

Este informe presenta un análisis de la complejidad computacional para las soluciones de dos problemas que se sacaron de la plataforma HackerRank: *Pairs* y *Sherlock and the Valid String*. Se examinan las transiciones desde enfoques ingenuos de fuerza bruta hacia estructuras de datos como *Hash Sets* y *Hash Maps*, demostrando matemáticamente cómo optimizar el rendimiento algorítmico para grandes volúmenes de datos.

1. Análisis de Complejidad: Problema *Pairs*

1.1. Definición del Problema

Dado un arreglo de N enteros únicos y un valor objetivo K , el algoritmo debe determinar el número de pares de enteros (a, b) cuya diferencia absoluta sea exactamente igual a K , es decir:

$$a - b = K \quad (1)$$

1.2. Enfoque Óptimo: Uso de *Hash Sets*

Podemos reescribir la ecuación (1) algebraicamente para buscar un elemento específico en función del elemento actual b , despejando a :

$$a = b + K \quad (2)$$

En lugar de iterar nuevamente sobre todo el arreglo para encontrar a , insertamos todos los elementos en un *Hash Set*. El algoritmo óptimo realiza los siguientes pasos:

1. **Construcción del Set:** Se insertan los N elementos del arreglo en el conjunto. Cada inserción en un *Hash Set* promedio toma un tiempo constante $O(1)$. Para N elementos, el tiempo es $O(N)$.
2. **Búsqueda Lineal:** Se itera a través de cada elemento b del arreglo. Para cada b , se verifica si el elemento complementario $(b + K)$ existe dentro del conjunto.

1.2.1. Cálculo de la Complejidad Temporal

La función del tiempo total $T(N)$ se desglosa matemáticamente como:

$$T(N) = T_{\text{construcción}} + T_{\text{búsqueda}} \quad (3)$$

$$T(N) = N \cdot O(1) + N \cdot O(1) = O(N) + O(N) = O(2N) \quad (4)$$

Aplicando las propiedades de la notación asintótica donde se descartan las constantes multiplicativas (en este caso 2):

$$T(N) = O(N) \quad (5)$$

1.2.2. Cálculo de la Complejidad Espacial

La memoria adicional requerida está determinada por el espacio que ocupa el *Hash Set*. Dado que el problema especifica que todos los enteros en el arreglo original son únicos, el conjunto almacenará exactamente N elementos. Por lo tanto, la complejidad espacial es:

$$S(N) = O(N) \quad (6)$$

2. Análisis de Complejidad: *Sherlock and the Valid String*

2.1. Definición del Problema

Una cadena de texto S se considera válida si todos los caracteres que aparecen en ella tienen exactamente la misma frecuencia. También se considera válida si se puede eliminar exactamente un carácter en una posición de la cadena para que los caracteres restantes tengan la misma frecuencia. El algoritmo debe retornar "YES" o "NO".

2.2. Desglose del Algoritmo Óptimo

Para resolver este problema de manera óptima, se utiliza una estrategia de doble mapeo de frecuencias (*Frecuencia de Frecuencias*):

1. Contar cuántas veces aparece cada carácter en la cadena S de longitud N .
2. Contar las frecuencias de los resultados obtenidos en el paso anterior.

2.3. Análisis de Operaciones Matemáticas

2.3.1. Paso 1: Conteo de Caracteres

Se recorre la cadena S de longitud N de principio a fin. El acceso e incremento del contador en un *HashMap* toma un tiempo $O(1)$.

$$T_{\text{paso1}}(N) = N \cdot O(1) = O(N) \quad (7)$$

El tamaño máximo de este primer mapa está estrictamente limitado por el tamaño del alfabeto utilizado. Si asumimos el alfabeto estándar en español de letras minúsculas (sin contar Ñ), el espacio máximo es una constante $|\Sigma| = 26$. Por lo tanto, la complejidad espacial de este mapa es $O(|\Sigma|) = O(26) = O(1)$ (Orden constante).

2.3.2. Paso 2: Conteo de Frecuencias de Frecuencias

Se extraen los valores numéricos del primer mapa (un máximo de 26 valores) y se mapean en un segundo diccionario para saber cuántas letras comparten la misma frecuencia. Dado que el tamaño del conjunto de entrada para este paso está acotado por la constante $|\Sigma|$, el tiempo de ejecución de esta etapa es:

$$T_{\text{paso2}} = O(|\Sigma|) = O(26) = O(1) \quad (8)$$

¡También es Orden constante!

2.3.3. Paso 3: Evaluación de Condiciones Límite

Una vez obtenido el mapa secundario, existen tres escenarios para determinar la validez del string:

- **Escenario 1:** La longitud del mapa secundario es 1. Significa que todas las letras tienen la misma frecuencia (Ej: $\text{abc} \rightarrow \text{frecuencias: } \{1: 3\}$). **Resultado: YES.**
- **Escenario 2:** La longitud es mayor que 2. Significa que hay 3 o más frecuencias distintas, lo cual no se puede solucionar quitando un solo carácter. **Resultado: NO.**

- **Escenario 3:** La longitud es exactamente 2. Sean $f_{\min}$ y $f_{\max}$ las dos frecuencias encontradas. Es válido si y solo si:

1. $f_{\min} = 1$ y solo una letra tiene dicha frecuencia (se elimina esa letra por completo).
2. $(f_{\max} - f_{\min}) = 1$ y solo una letra tiene la frecuencia máxima $f_{\max}$ (se le resta un carácter a esa letra para igualarla al resto).

Cualquiera de estas comprobaciones se realiza sobre estructuras de tamaño máximo igual a 2, por lo que toma tiempo constante $O(1)$.

2.4. Complejidad Total del Algoritmo

Sumando los pasos temporales:

$$T_{\text{total}}(N) = O(N) + O(1) + O(1) = O(N) \quad (9)$$

Respecto al espacio, las estructuras no crecen de manera proporcional a N , sino que están limitadas por el alfabeto $|\Sigma|$:

$$S_{\text{total}}(N) = O(|\Sigma|) = O(26) = O(1) \quad (10)$$

3. Discusión y Resumen Comparativo

La selección adecuada de estructuras de datos altera drásticamente la viabilidad de una solución. El uso de *Hash Maps* y *Sets* permite transformar problemas que exponencialmente se vuelven intratables (Como $O(N^2)$) en soluciones de tiempo lineal ($O(N)$), las cuales pueden procesar millones de registros en milisegundos.

Cuadro 1: Tabla Comparativa de Complejidades

Problema	Tiempo (Óptimo)	Espacio (Óptimo)
Pairs	$O(N)$	$O(N)$
Sherlock String	$O(N)$	$O(1)$

A través del análisis realizado en este informe, queda demostrado que ambos problemas de Hacke-rRank se resuelven de forma óptima en tiempo lineal respecto al tamaño de la entrada.